

MIDIsynthesis: генерация MIDI-музыки с помощью Transformer

1. Краткое описание проекта

Цель проекта — разработать систему генерации MIDI-музыки с помощью нейросетевой модели типа decoder-only Transformer. Модель обучается на последовательностях музыкальных токенов и решает задачу предсказания следующего токена. После обучения модель может генерировать новые последовательности, которые затем конвертируются обратно в MIDI и воспроизводятся через веб-интерфейс.

2. Постановка задачи

Исходная задача формулируется как autoregressive next-token prediction.

На вход модель получает последовательность музыкальных токенов:

START D5_0.5 E5_0.5 F#5_1.0

и должна предсказать следующий токен.

После этого предсказанный токен добавляется к контексту, и процесс повторяется до достижения END или лимита длины генерации.

3. Данные

В проекте используется Nottingham MIDI Dataset.

MIDI-файлы были преобразованы в текстовые музыкальные токены. Каждый токен описывает либо одну ноту, либо аккорд с длительностью.

Примеры токенов:

Токен	Значение
D5_0.5	нота D5 длительностью 0.5
D2_F#2_A2_4.0	аккорд D-major длительностью 4.0
START	начало последовательности
END	конец последовательности

4. Архитектура модели

В проекте используется decoder-only Transformer, похожий по принципу на GPT-подобные модели, но применённый к музыкальным токенам.

Основные компоненты модели:

- token embedding;
- positional embedding;
- multi-head self-attention;
- feed-forward блоки;
- residual connections;
- layer normalization;
- linear output head.

Модель предсказывает распределение вероятностей по словарю музыкальных токенов.

5. Обучение

Функция потерь: Cross Entropy Loss.

Оптимизатор: AdamW.

Для логирования экспериментов использовался ClearML. В ClearML сохранялись:

- train loss;
- validation loss;
- гиперпараметры;
- checkpoint модели;
- финальная модель;
- артефакты генерации.

6. Исправление train/validation leakage

Изначально использовался random_split поверх уже созданного sliding-window dataset. Это приводило к утечке данных: соседние окна могли попадать одновременно в train и validation.

Проблема была исправлена: разделение теперь выполняется до создания окон.

Было:

```
dataset = MIDIDataset(token_ids, block_size)
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])
```

Стало:

```
split_idx = int(0.9 * len(token_ids))

train_tokens = token_ids[:split_idx]
val_tokens = token_ids[split_idx:]

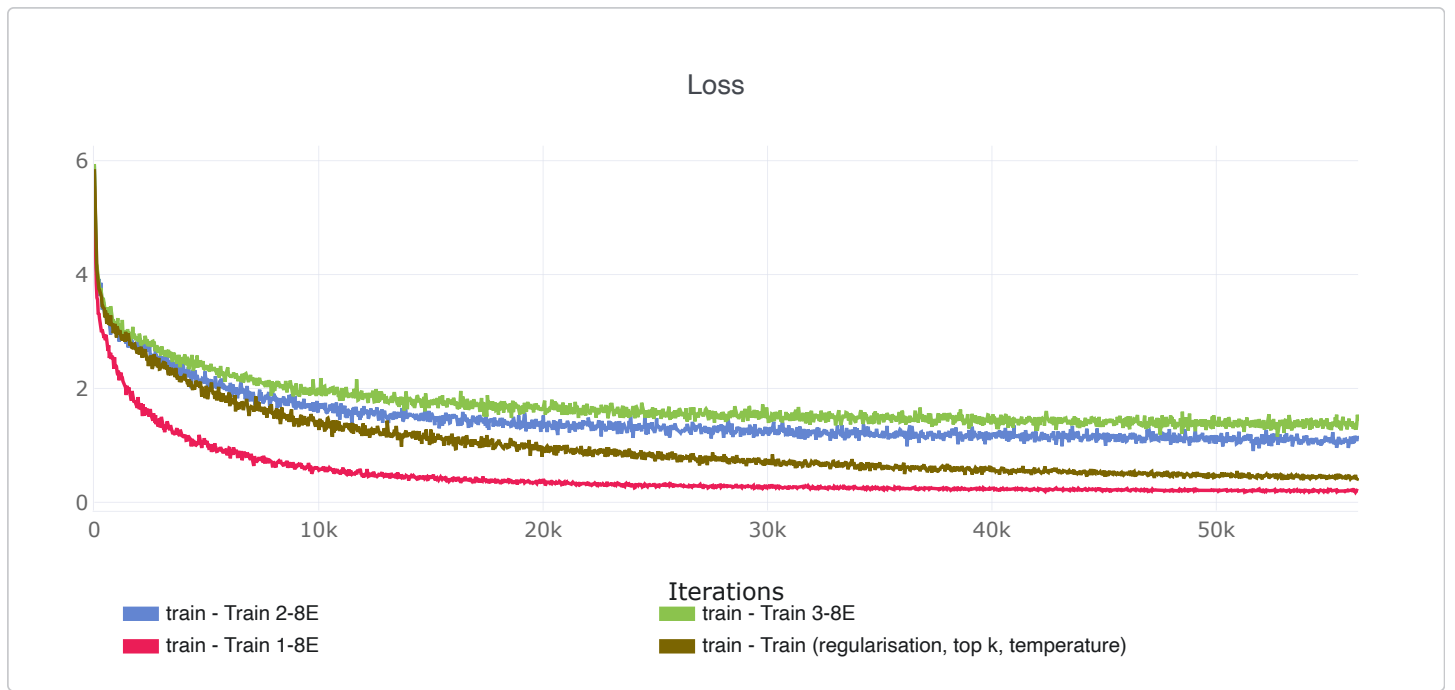
train_dataset = MIDIDataset(train_tokens, block_size)
val_dataset = MIDIDataset(val_tokens, block_size)
```

7. Эксперименты

Было обучено несколько конфигураций модели.

Модель	d_model	n_heads	n_layers	dropout	weight_decay	Цель эксперимента	Результат
Baseline Transformer	256	8	4	0.2	0	Получить базовое качество генерации	Хороший loss, но заметные повторы
Compact Transformer v1	128	4	3	0.3	1e-2	Снизить переобучение	Более лёгкая и стабильная модель
Compact Transformer v2	128	4	2	0.3	1e-2	Проверить минимальную архитектуру	Быстрее обучается, но может быть менее выразительной
Regularized Transformer	256	8	4	0.3–0.4	1e-2	Сохранить мощность модели и усилить регуляризацию	Лучший кандидат при хорошем балансе loss/генерации

Сравнение loss по экспериментам



8. Метрики и критерии оценки

Для оценки обучения использовались количественные метрики:

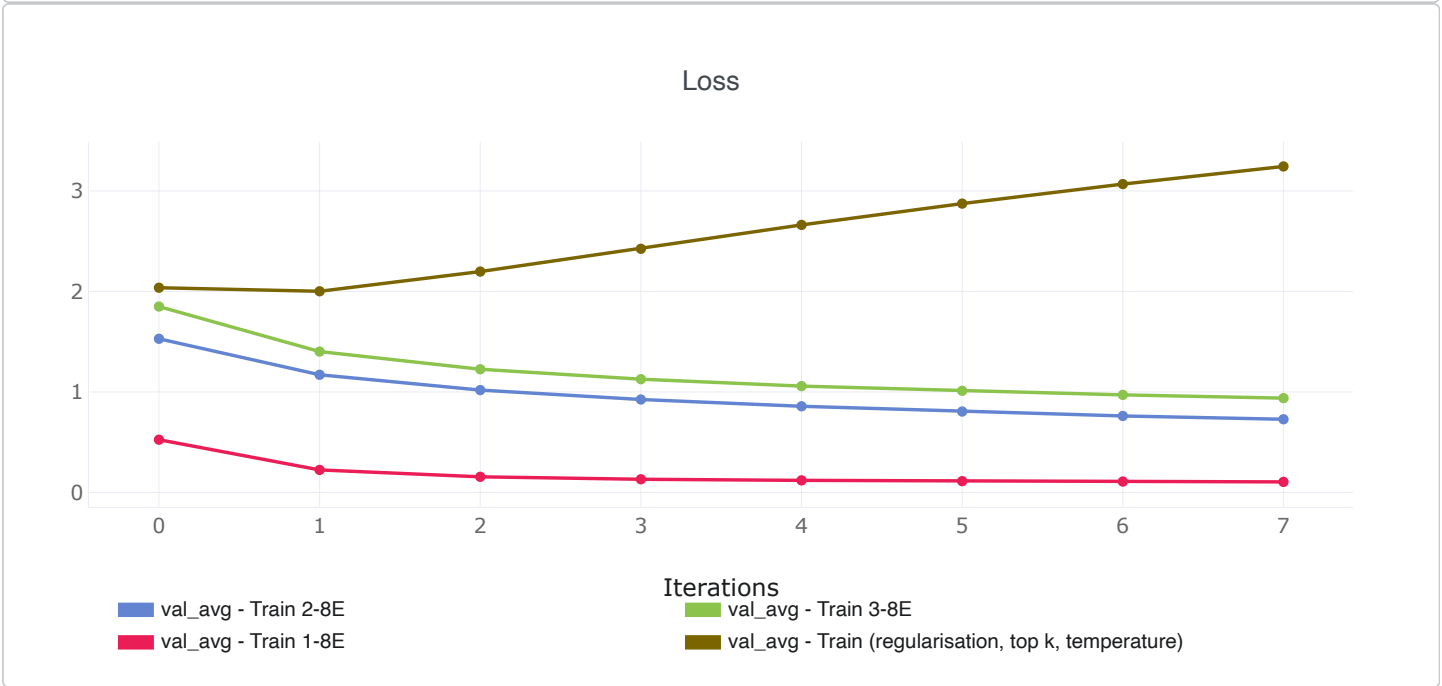
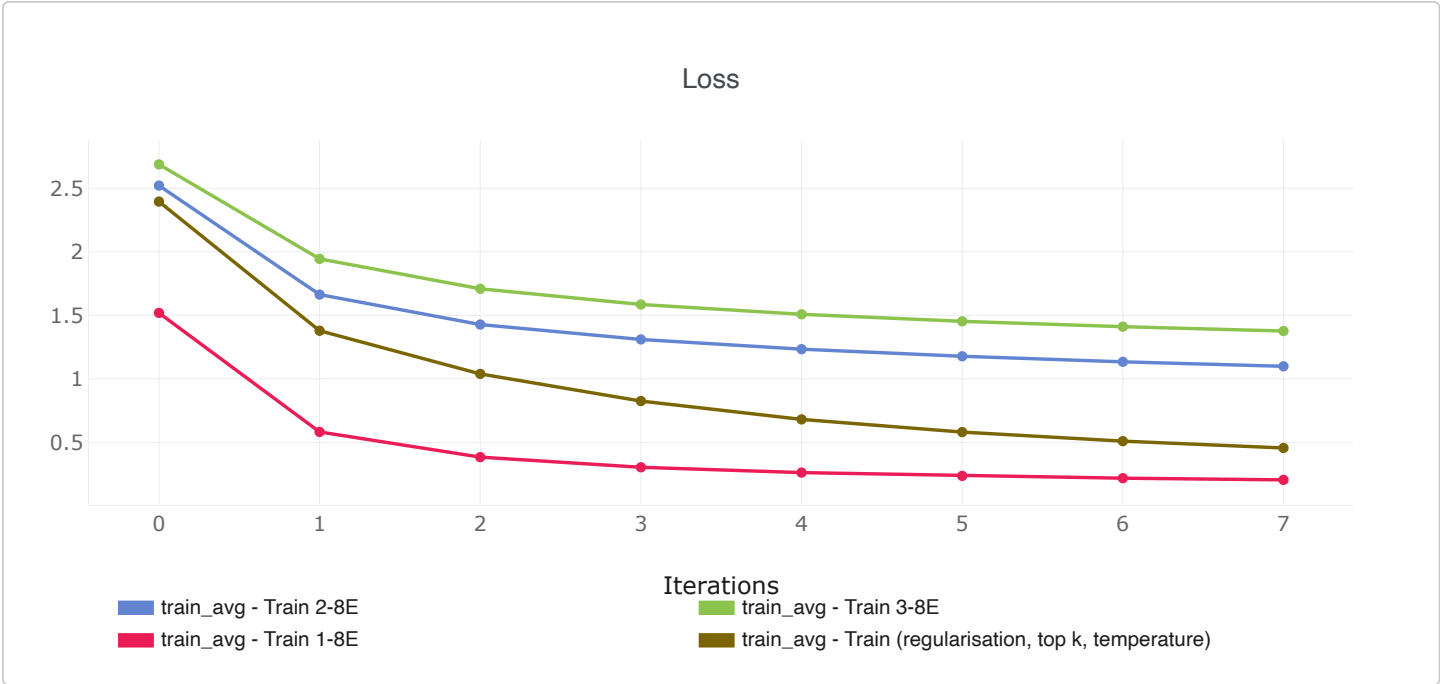
- `train_loss`;
- `val_loss`.

Для оценки качества генерации использовались качественные критерии:

- наличие музыкальной структуры;
- повторяемость фраз;
- разнообразие при разных запусках;
- субъективное качество звучания;
- наличие завершения последовательности токеном `END`.

Repetition rate как отдельная численная метрика в текущей версии проекта не рассчитывался. Вместо этого повторяемость оценивалась по сгенерированным токен-последовательностям и MIDI-файлам.

9. ClearML-графики



10. Сравнение моделей

На основе графиков `train_avg` и `val_avg` были сравнены четыре обученные модели.

Модель	Train loss	Validation loss	Интерпретация
Train 1-8E	Самый низкий	Самый низкий	Лучшая модель по validation loss. Быстро обучается и показывает стабильное снижение ошибки на validation set.
Train 2-8E	Средний	Средний	Стабильная модель без явных признаков переобучения, но уступает Train 1-8E.
Train 3-8E	Самый высокий	Выше, чем у Train 2-8E	Обучается стабильно, но показывает худшее качество среди стабильных моделей.
Train regularisation, top-k, temperature	Снижается	Растёт	Признаки переобучения: train loss падает, но validation loss увеличивается.

По результатам сравнения лучшей моделью была выбрана **Train 1-8E**, так как она показывает минимальный `val_avg` и стабильную динамику обучения.

Эксперимент **Train (regularisation, top k, temperature)** показал классический признак переобучения: ошибка на обучающей выборке снижается, но ошибка на validation set растёт. Поэтому эта модель не была выбрана как финальная, несмотря на снижение train loss.

Итоговый рейтинг моделей по validation loss:

1. **Train 1-8E** — лучший результат;
2. **Train 2-8E** — стабильный baseline;
3. **Train 3-8E** — стабильная, но менее качественная модель;
4. **Train (regularisation, top k, temperature)** — переобучение.

11. Генерация

Для генерации используется autoregressive sampling.

Параметры генерации:

Параметр	Значение
temperature	контролирует случайность генерации
top_k	ограничивает выбор K наиболее вероятными токенами
max_new_tokens	максимальная длина генерации
start_token	начальный prompt

Пример параметров:

```
temperature = 1.8
top_k = 100
max_new_tokens = 256
```

12. Проблемы и решения

Проблема 1: одинаковые генерации

При одинаковом prompt модель часто генерировала очень похожие последовательности.

Причины:

- маленький и повторяющийся датасет;
- высокая уверенность модели;
- фиксированный начальный prompt;
- возможное переобучение.

Решения:

- добавлен temperature sampling;
- добавлен top-k sampling;
- использованы разные начальные prompts;
- уменьшено число эпох;
- добавлены dropout и weight decay;
- исправлен train/validation split.

Проблема 2: воспроизведение MIDI в браузере

Браузерный аудио-плеер не поддерживает воспроизведение MIDI напрямую, так как MIDI — это не аудиофайл, а набор музыкальных команд (по сути - текст).

Решение:

MIDI конвертируется в WAV → WAV воспроизводится через стандартный браузерный audio player

13. Веб-интерфейс

В проект добавлен FastAPI backend и HTML-интерфейс.

Возможности интерфейса:

- запуск генерации;
- ввод seed;
- настройка temperature;
- прослушивание WAV;
- скачивание MIDI;
- просмотр истории генераций.

Запуск:

```
python web/main.py
```

После запуска интерфейс доступен по адресу:

`http://localhost:8000/static/index.html`

14. Результаты

В результате был реализован полный pipeline:

1. загрузка MIDI-данных;
2. токенизация;
3. обучение Transformer-модели;
4. логирование экспериментов в ClearML;
5. сохранение checkpoint;
6. генерация новых MIDI;
7. конвертация MIDI в WAV;
8. веб-интерфейс для демонстрации.

Лучшая модель: **Train 1-8E**

15. Ограничения

Основные ограничения проекта:

- небольшой размер Nottingham dataset;
- генерации иногда повторяют похожие музыкальные фразы;
- качество зависит от prompt и параметров sampling;
- MIDI-токенизация упрощает музыкальную структуру.

16. Возможные улучшения

В будущем можно улучшить проект следующим образом:

- использовать больший MIDI dataset;
- добавить более богатую токенизацию;
- использовать repetition penalty;
- добавить top-p sampling;
- обучить модель на более длинном контексте;
- добавить выбор инструмента в веб-интерфейсе;
- улучшить качество синтеза через FluidSynth и SoundFont.